TECNICATURA UNIVERSITARIA

EN PROGRAMACIÓN

A DISTANCIA

Trabajo Práctico – Virtualización con VirtualBox

Alumno:

Jalil, Enzo Luciano – jenzoluciano@gmail.com

Materia: Arquitectura y Sistemas Operativos

Profesor/a: Costello Patricio

Fecha de entrega: 09/11/2026

Link al repositorio: https://github.com/dakucrow/tpiAySO2026

(En el README del repositorio se encuentra el link directo al video de presentación y con las capturas de pantalla con su evidencia)

Índice:

1. Introducción
2. Marco Teórico
3. Caso Práctico
4. Metodología Utilizada
5. Resultados Obtenidos
6. Conclusiones
7. Bibliografía
8. Anexos

1. Introducción

El presente trabajo aborda la implementación práctica de la virtualización de recursos informáticos y la administración de sistemas operativos mediante la interfaz de línea de comandos (CLI). La elección de este tema responde a la necesidad de comprender en profundidad cómo interactúan los componentes físicos de una computadora con los sistemas operativos huésped, y cómo se aprovisionan entornos aislados y seguros para el despliegue de aplicaciones en la actualidad.

Para la formación de un Técnico en Programación, el desarrollo de esta experiencia resulta de gran importancia profesional. En el ámbito laboral real, la gran mayoría de las aplicaciones se ejecutan en servidores remotos que se administran de manera exclusiva a través de la consola de comandos, sin

interfaz gráfica. Aprender a crear y configurar una máquina virtual en VirtualBox, instalar un sistema operativo de servidor, gestionar el código fuente con Git y GitHub, y aplicar mecanismos de respaldo mediante instantáneas (snapshots), aporta las herramientas indispensables para desenvolverse en entornos reales de desarrollo y despliegue de software.

Con el propósito de llevar a cabo esta experiencia, el trabajo se propone como objetivo general crear, configurar y validar un entorno virtualizado seguro utilizando el hipervisor Oracle VM VirtualBox para la instalación de Ubuntu Server 24.04 LTS, la ejecución de un programa en Python desplegado desde GitHub y la aplicación de políticas de respaldo. Para alcanzar esta meta, se aborda de manera secuencial la asignación adecuada de vCPU, memoria RAM y almacenamiento secundario dinámico (VDI), la parametrización del sistema operativo en modo consola habilitando el servicio OpenSSH para la administración remota, la instalación de la herramienta Git mediante el gestor de paquetes apt para clonar el repositorio del proyecto, la verificación práctica del script imparPar.py dentro del entorno Linux y, finalmente, la generación de puntos de restauración mediante instantáneas que garanticen la tolerancia a fallos y la continuidad operativa del servidor ante cualquier contingencia.

2. Marco Teórico:

El soporte conceptual de este proyecto abarca los principios fundamentales de la virtualización de recursos, la clasificación de hipervisores, el funcionamiento de la memoria y procesador virtual, la administración por consola en Linux y la gestión de servicios remotos.

2.1. Fundamentos de Virtualización e Hipervisores

La virtualización es la tecnología que permite abstraer los recursos físicos de una computadora (CPU, memoria RAM, dispositivos de almacenamiento y adaptadores de red) para presentarlos como entornos computacionales independientes denominados **máquinas virtuales (VMs)**. La virtualización resuelve la ineficiencia de dedicar hardware físico exclusivo a un solo servicio, permitiendo consolidar múltiples sistemas en un mismo equipo con aislamiento total.

La pieza central de esta arquitectura es el **Hipervisor** o *Virtual Machine Monitor* (VMM), un componente de software que actúa como intermediario entre el hardware subyacente y los sistemas operativos invitados (*Guest OS*). La literatura técnica distingue dos categorías principales de hipervisores:

- **Hipervisor Tipo 1 (Nativo o Bare-metal):** Se ejecuta directamente sobre el hardware físico sin un sistema operativo anfitrión intermedio. Ejemplos de esta categoría incluyen VMware ESXi, Microsoft Hyper-V y Proxmox VE. Ofrecen un alto rendimiento y baja latencia, siendo la opción predilecta para centros de datos y servidores de producción.
- **Hipervisor Tipo 2 (Alojado o Hosted):** Se ejecuta como una aplicación sobre un sistema operativo anfitrión (*Host OS*) ya existente (como

Windows o macOS). Ejemplos representativos son Oracle VM VirtualBox y VMware Workstation. Aunque presentan una ligera penalización en el rendimiento debido a la capa adicional del SO anfitrión, ofrecen una flexibilidad excepcional para entornos de desarrollo, laboratorios y pruebas.

Criterio de Comparación	Hipervisor Tipo 1 (Bare-Metal)	Hipervisor Tipo 2 (Alojado)
Ubicación en la pila	Directamente sobre el hardware	Sobre el Sistema Operativo Anfitrión
Acceso al hardware	Directo mediante controladores propios	Indirecto a través de las APIs del SO anfitrión
Rendimiento	Nativo (muy baja latencia)	Penalización por la capa del SO anfitrión
Escenario principal	Centros de Datos, Nube, Producción	Entornos de prueba, laboratorios, desarrollo
Ejemplo representativo	VMware ESXi, Proxmox VE	Oracle VM VirtualBox, VMware Workstation

2.2. Virtualización de Recursos: CPU, Memoria, Almacenamiento y Red

El hipervisor de Tipo 2 implementa técnicas de abstracción para gestionar los componentes de hardware asignados a la máquina virtual:

1. **Virtualización de CPU y vCPU:** Las unidades virtuales de procesamiento (vCPU) permiten al hipervisor programar hilos de ejecución en los núcleos físicos del procesador. Se utilizan extensiones de virtualización por hardware (**Intel VT-x** / **AMD-V**) para ejecutar instrucciones de forma segura sin interferir con el kernel anfitrión.
2. **Virtualización de Memoria (vRAM):** El hipervisor asigna un bloque de memoria RAM del anfitrión a la máquina virtual y gestiona la traducción de direcciones virtuales a direcciones físicas del sistema mediante tecnologías de traducción de direcciones de segundo nivel (**SLAT**).
3. **Virtualización de Almacenamiento (Archivos VDI, ISO y Snapshots):**
    - **Archivo VDI (Virtual Disk Image):** Es el formato de disco virtual propio de VirtualBox que representa el disco rígido de la máquina virtual como un archivo contenedor dinámico en el sistema de archivos del anfitrión.
    - **Imagen ISO:** Es un archivo de imagen que emula un disco óptico conteniendo la instalación de **Ubuntu Server 24.04 LTS**, el cual

se "monta" en la lectora virtual de la VM para iniciar el proceso de instalación.

    - **Instantáneas (Snapshots):** Mecanismo de seguridad que congela el estado del disco virtual base (.vdi) en modo solo lectura y genera un **disco diferencial** para almacenar las modificaciones posteriores. Esto permite restaurar el servidor a un punto anterior exacto ante cualquier falla o contingencia.

4. **Virtualización de Red (Modos de Red y NAT):** El hipervisor crea tarjetas de red virtuales (vNIC). En el entorno de VirtualBox, el modo por defecto **NAT (Network Address Translation)** permite que la máquina virtual comparta la conexión a Internet del sistema anfitrión a través de una dirección IP privada asignada por un servidor DHCP interno de VirtualBox, facilitando la descarga de paquetes y clonado de repositorios sin exponer la VM a la red física local.

2.3. Comparativa: Máquinas Virtuales vs. Contenedores (Docker)

Durante el estudio de la arquitectura de sistemas se contrapone la virtualización tradicional basada en máquinas virtuales con la virtualización a nivel de sistema operativo basada en contenedores (como **Docker**):

- **Máquina Virtual (VirtualBox):** Virtualiza todo el hardware físico. Cada máquina virtual ejecuta un sistema operativo completo con su propio kernel de Linux o Windows, lo que proporciona un aislamiento total a costa de un mayor consumo de recursos (RAM y disco) y mayor tiempo de arranque.
- **Contenedor (Docker):** Virtualiza únicamente el sistema operativo. Los contenedores compartirán el mismo kernel del sistema operativo anfitrión y empaquetan únicamente la aplicación junto a sus dependencias, resultando en un entorno de ejecución extremadamente liviano y de rápido inicio.

2.4. Administración por Consola (CLI), Gestor de Paquetes (APT) y Control de Versiones (Git)

- **Interfaz de Línea de Comandos (CLI):** Administración del servidor mediante comandos de texto sin interfaz gráfica, optimizando los recursos de procesamiento y RAM del equipo.
- **Gestor de Paquetes (apt):** Herramienta de Ubuntu utilizada para actualizar repositorios (apt update) e instalar herramientas del sistema (apt install) resolviendo dependencias de software automáticamente.
- **Control de Versiones (git):** Sistema distribuido que permite descargar e interconectar el código fuente del proyecto (imparPar.py) desde un repositorio público alojado en GitHub hacia el servidor virtual mediante comandos de clonación (git clone).

3. Caso Práctico

En esta sección se presenta la situación práctica desarrollada para aplicar los contenidos de la materia, combinando la programación, la virtualización de servidores y el uso de Git y GitHub.

3.1. Problema a Resolver

El problema a resolver consistió en crear un programa sencillo en Python (imparPar.py) que solicita al usuario ingresar números enteros por teclado, los clasifica en dos grupos (pares e impares) y calcula la suma total de cada grupo. Para evitar que el programa falle si se ingresa una letra o un texto por error, se incluyó un manejo de excepciones con bloques try/except.

Una vez verificado su funcionamiento en la PC, el siguiente paso fue trasladar el código a un servidor Linux sin interfaz gráfica instalado en la máquina virtual de VirtualBox. Para realizar esta transferencia de forma profesional, se subió el código a GitHub y se descargó dentro del servidor usando Git. Finalmente, se generó una instantánea (snapshot) en VirtualBox para guardar un punto de respaldo de toda la configuración.

3.2. Estructura de Evidencias y Capturas de Pantalla

La evidencia técnica del trabajo práctico se organizó de manera sistemática en cuatro carpetas de capturas digitales, reflejando el progreso cronológico del proyecto:

capturas/

├── 01_generacion_de_codigo/

│ ├── 01_solicitud_codigo_base

│ ├── 02_Agregar_validaciones

│ ├── 03_Pruebas_de_codigo

│ └── 04_subida_git

├── 02_creacion_vm/

│ ├── 05_Seleccion_ISO

│ ├── 06_Ram_CPU

│ ├── 07_Disco

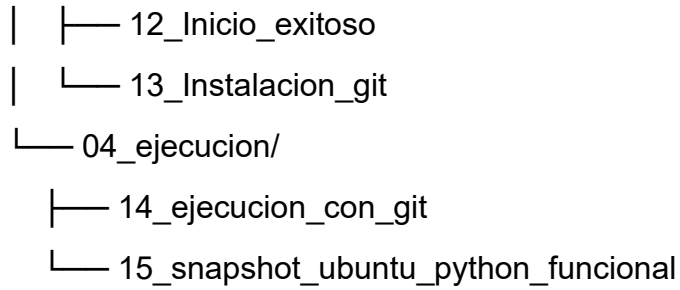
│ └── 08_Generacion_terminada

├── 03_instalacion/

│ ├── 09_Configuracion_perfil

│ ├── 10_OpenSSH

│ ├── 11_Instalacion_completa



3.3. Validación del Funcionamiento

La validación operativa del caso práctico se concretó exitosamente mediante los siguientes hitos verificados en la consola del servidor:

1. **Aprovisionamiento:** Creación de la VM con 2 vCPU, 2048 MB de RAM y disco VDI de 25 GB.
2. **Conectividad e Instalación:** Instalación limpia de Ubuntu Server 24.04 LTS con selección explícita del servidor OpenSSH y configuración de red mediante DHCP.
3. **Despliegue de Código:** Actualización de catálogos con sudo apt update, instalación del cliente Git con sudo apt install -y git y clonación del proyecto desde GitHub.
4. **Ejecución y Portabilidad:** Invocación del script con python3 src/imparPar.py. El intérprete nativo de Python 3 en Ubuntu procesó las entradas, clasificó los números pares e impares y calculó los totales correctamente sin requerir dependencias adicionales.
5. **Tolerancia a Fallos:** Generación de la instantánea Estado_Inicial_Script_Python en VirtualBox, congelando el estado del disco y la memoria para garantizar la recuperación ante fallas.

4. Metodología Utilizada

El proyecto se ejecutó siguiendo una metodología estructurada en seis fases secuenciales, combinando investigación documental, desarrollo de código y pruebas prácticas.

Detalle de las Etapas de Trabajo

1. **Fase 1: Desarrollo y Pruebas Locales (Windows):** Escribir el script imparPar.py en VS Code incorporando estructuras defensivas try/except para prevenir caídas por datos inválidos. Validar la ejecución en el entorno local.
2. **Fase 2: Gestión de Código con Git y GitHub:** Inicializar el repositorio Git local, configurar la carpeta src/, realizar los commits correspondientes y publicar el proyecto en un repositorio público de GitHub.

3. **Fase 3: Aprovisionamiento del Hardware Virtual (VirtualBox):** Configurar el hipervisor Tipo 2 en el sistema anfitrión. Crear una nueva VM asignando 2048 MB de vRAM, 2 procesadores virtuales (vCPU) y un disco virtual VDI de 25 GB.
4. **Fase 4: Instalación y Endurecimiento del Servidor:** Asociar la imagen ISO oficial de Ubuntu Server 24.04 LTS a la unidad óptica virtual. Omitir la instalación desatendida para configurar manualmente el teclado, las particiones de almacenamiento, el usuario del sistema y habilitar el paquete de administración remota **OpenSSH Server**.
5. **Fase 5: Despliegue y Ejecución en Linux:** Iniciar sesión en el sistema sin interfaz gráfica. Ejecutar sudo apt update && sudo apt install -y git para obtener el gestor de versiones. Clonar el repositorio con git clone, navegar al directorio con cd y ejecutar el programa con python3 imparPar.py.
6. **Fase 6: Respaldo y Tolerancia a Fallos (Snapshot):** Liberar la consola mediante la tecla Host (Ctrl Derecha) y tomar la instantánea desde la interfaz de VirtualBox para registrar la persistencia del sistema.

5. Resultados Obtenidos

El desarrollo del caso práctico arrojó resultados ampliamente satisfactorios, cumpliendo con la totalidad de los requisitos técnicos planteados por la cátedra.

5.1. Logros Técnicos Verificados

- **Aislamiento y Eficiencia:** La máquina virtual operó de manera fluida consumiendo únicamente los recursos asignados (2 GB RAM), manteniendo un aislamiento total con respecto al sistema operativo Windows anfitrión.
- **Despliegue de Código Transparente:** La clonación a través de HTTPS mediante Git permitió transferir la aplicación en segundos, demostrando que la separación entre el código fuente y el entorno de ejecución simplifica el despliegue de software.
- **Disponibilidad de Intérprete Nativo:** Se constató que Ubuntu Server incluye Python 3 por defecto en su instalación base, lo que eliminó la necesidad de instalar dependencias pesadas y garantizó una ejecución inmediata.
- **Persistencia Asegurada:** La prueba de restauración de la instantánea (*snapshot*) demostró la capacidad de devolver la VM a su estado completamente funcional en menos de 3 segundos ante cualquier eventual falla del sistema.

5.2. Dificultades Presentadas y Soluciones Aplicadas

Durante el desarrollo del trabajo práctico se presentaron algunos inconvenientes técnicos menores que fueron resueltos mediante la aplicación de comandos y configuraciones en el hipervisor:

1. **Mensaje de advertencia al reiniciar (Failed unmounting /cdrom):** Al finalizar la instalación de Ubuntu Server y reiniciar la máquina virtual, apareció en pantalla un mensaje indicando que no se podía desmontar el disco de instalación. Esto ocurrió porque VirtualBox retiene la imagen ISO montada en la unidad lectora virtual durante el primer reinicio. Para solucionarlo, simplemente se presionó la tecla **Enter** en la consola, lo que indicó al hipervisor que liberara la imagen ISO y permitiera iniciar el sistema operativo normalmente desde el disco VDI.
2. **Captura del teclado por parte de la máquina virtual:** Al interactuar con la terminal de Linux, VirtualBox tomó el foco de entrada del teclado y del mouse, lo que impedía utilizar atajos de teclado propios de Windows (como Win + Shift + S para realizar capturas de pantalla de las evidencias). La solución consistió en presionar la **Tecla Host**, la cual libera el foco de la máquina virtual y devuelve el control del teclado al sistema operativo anfitrión.
3. **Falta del paquete Git en la instalación base:** Al intentar clonar el repositorio del proyecto desde GitHub, la consola devolvió un mensaje indicando que el comando git no existía. Esto se debe a que Ubuntu Server realiza una instalación mínima para ahorrar recursos y espacio en disco, dejando de lado herramientas secundarias. Se resolvió ejecutando la actualización de repositorios con sudo apt update e instalando la herramienta mediante sudo apt install -y git a través del gestor de paquetes APT.

7. Conclusiones

La realización de este trabajo integrador permitió poner en práctica los conceptos vistos a lo largo de la materia Arquitectura y Sistemas Operativos, pasando de la teoría a la configuración de un servidor real.
En primer lugar, se comprendió la utilidad de los hipervisores Tipo 2 como VirtualBox. Si bien tienen una pequeña pérdida de rendimiento en comparación con los servidores que van directo sobre el hardware (Tipo 1), son la mejor alternativa para crear entornos de prueba seguros y aprender a configurar sistemas operativos sin poner en riesgo la computadora personal.
En segundo lugar, se comprobó la importancia de administrar servidores a través de la consola de comandos (CLI). Trabajar sin interfaz gráfica permite ahorrar una gran cantidad de memoria RAM y uso de procesador, dejando esos recursos disponibles para que las aplicaciones funcionen mejor. Además, esta forma de trabajo refleja el día a día profesional, donde los servidores remotos se gestionan exclusivamente mediante comandos.

Por último, el proyecto demostró cómo se conectan las distintas herramientas del desarrollo actual: el uso de Python para crear el programa, Git y GitHub para trasladar el código de forma rápida desde la PC hacia el servidor, y las instantáneas (snapshots) de VirtualBox para guardar puntos de respaldo por si algo falla. En conjunto, esta experiencia permitió entender el camino completo desde que se programa una solución hasta que se la deja funcionando en un servidor.

8. Bibliografía

Las fuentes documentales y bibliográficas consultadas para la fundamentación teórica y desarrollo del proyecto se detallan a continuación:

- **Adams, K., & Agesen, O. (2006).** *A Comparison of Software and Hardware Techniques for x86 Virtualization.* ACM SIGOPS Operating Systems Review, 40(5), 142-153.
- **Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018).** *Operating Systems: Three Easy Pieces.* Arpaci-Dusseau Books. Apéndice "Virtual Machine Monitors".
- **Bugnion, E., Devine, S., Rosenblum, M., Sugerman, J., & Wang, E. (2012).** *Bringing Virtualization to the x86 Architecture with the Original VMware Workstation.* ACM Transactions on Computer Systems (TOCS), 30(4), 1-51.
- **Oracle Corporation. (2024).** *Oracle VM VirtualBox User Manual (Version 7.0).* Documentación oficial de Oracle.
- **Popek, G. J., & Goldberg, R. P. (1974).** *Formal Requirements for Virtualizable Third Generation Architectures.* Communications of the ACM, 17(7), 412-421.
- **Red Hat, Inc. (2024).** *¿Qué es un hipervisor? Conceptos de Virtualización y Contenedores.* Red Hat Customer Portal.
- **Silberschatz, A., Galvin, P. B., & Gagne, G. (2018).** *Operating System Concepts* (10th ed.). John Wiley & Sons. Capítulo 16, "Virtual Machines".
- **Stallings, W. (2023).** *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.
- **Universidad Tecnológica Nacional (UTN). (2026).** *Unidad 7: Virtualización de Recursos por un Hipervisor Tipo 2.* Tecnicatura Universitaria en Programación a Distancia.